

DOKUMENTACJA PROJEKTOWA

System Zarządzania Kliniką Weterynaryjną (REST API + SPA)

Autor: Kamil Kędra

Uczelnia: Politechnika Krakowska

Kierunek: Informatyka

1. Wstęp i Cel Projektu

Celem projektu było zaprojektowanie oraz implementacja systemu informatycznego wspomagającego zarządzanie procesami w klinice weterynaryjnej. Aplikacja miała za zadanie zautomatyzować obsługę wizyt, ustandaryzować proces rejestracji pacjentów oraz zabezpieczyć harmonogram pracy personelu medycznego przed konfliktami wynikającymi z nałożenia się terminów rezerwacji. Realizacja opiera się na architekturze klient-serwer, gdzie backend wystawia interfejs REST API, z którym komunikuje się lekki klient webowy (Single Page Application).

2. Wykorzystany Stos Technologiczny

W projekcie zastosowano nowoczesne i powszechnie używane w branży IT rozwiązania narzędziowe i architektoniczne:

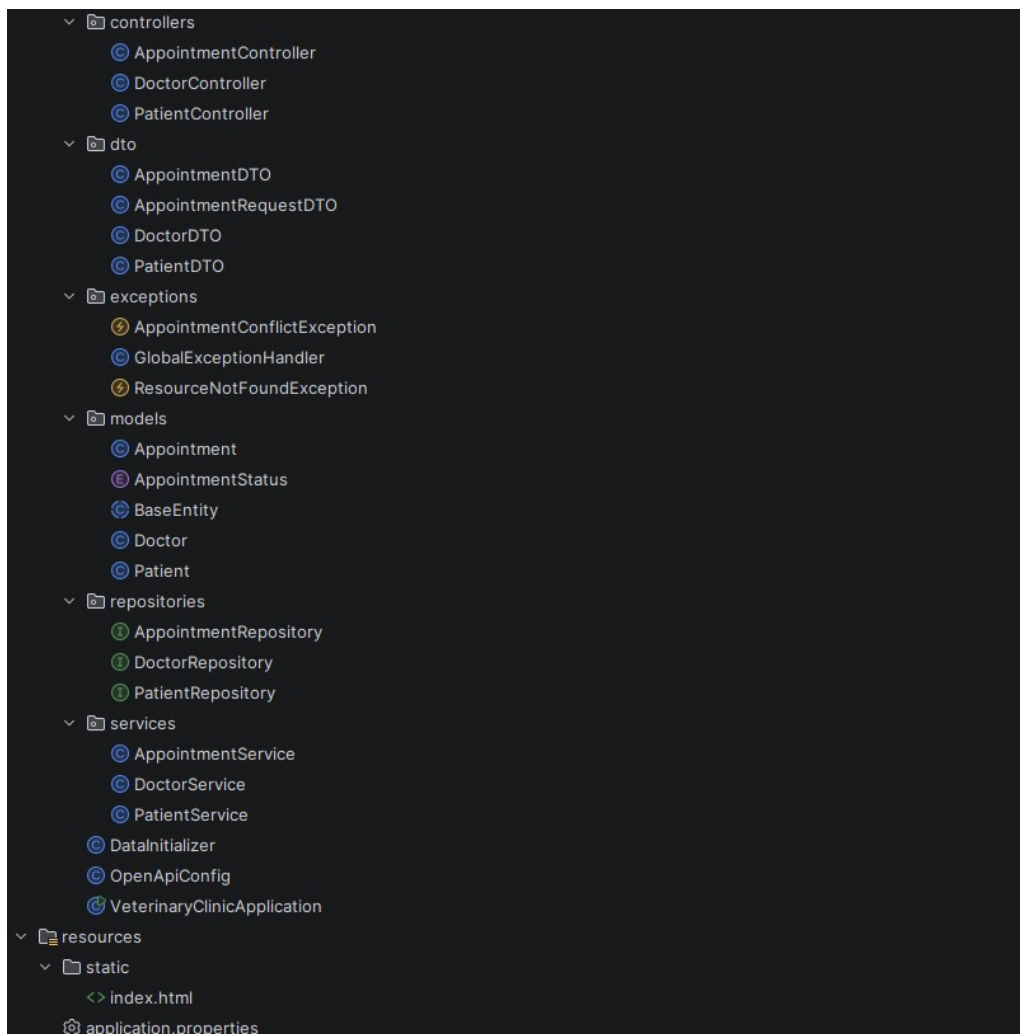
- **Backend:** Java 17, Spring Boot 3.2.5 (Spring Web, Spring Boot Validation).
- **Warstwa Persystencji:** Spring Data JPA (Hibernate) połączona z relacyjną bazą danych SQLite. Wybór SQLite podyktowany był chęcią stworzenia aplikacji przenośnej, niewymagającej konfiguracji zewnętrznego serwera bazy danych.
- **Frontend:** Vanilla HTML5, CSS3, JavaScript z wykorzystaniem asynchronicznego Fetch API do komunikacji z serwerem.
- **Testowanie:** Środowisko JUnit 5 połączone z frameworkiem Mockito oraz biblioteką AssertJ, co pozwoliło na dokładną izolację testowanych komponentów.
- **Dokumentacja API:** Springdoc OpenAPI (Swagger UI) do dynamicznego generowania specyfikacji endpointów.

3. Architektura Systemu i Wzorce Projektowe

System zrealizowano zgodnie z paradygmatem MVC (Model-View-Controller) oraz architekturą trójwarstwową, zapewniającą wysoką spójność i luźne powiązania pomiędzy komponentami:

- **Warstwa Kontrolerów (Controllers):** Odpowiada za przyjmowanie żądań HTTP, walidację danych wejściowych i zwracanie odpowiednich kodów statusu (np. 200, 201, 400).
- **Warstwa Usług (Services):** Hermetyzuje kluczową logikę biznesową. To tutaj podejmowane są decyzje o tym, czy operacja może zostać wykonana na podstawie reguł dziedzicznych.
- **Warstwa Dostępowa (Repositories):** Interfejsy rozszerzające JpaRepository, odpowiadające za abstrakcję zapytań SQL.

Dodatkowo w projekcie wykorzystano wzorzec **DTO (Data Transfer Object)**. Oddziela on fizyczną strukturę tabel bazy danych (Encje) od struktury danych wysyłanych do klienta, co znacząco zwiększa bezpieczeństwo aplikacji i zapobiega wyciekowi wrażliwych informacji.



Rys. 1. Struktura projektu w IntelliJ IDEA, pokazuje podział na pakiety zgodnie z architekturą (kontrolery, serwisy, repozytoria, modele).

4. Baza Danych i Mechanizm Audytu

Model danych opiera się na trzech głównych encjach powiązanych ze sobą relacjami (Lekarz, Pacjent, Wizyta).

Aby zapewnić pełną identyfikowalność operacji, w systemie zaimplementowano mechanizm JPA Auditing. Każda encja dziedziczy po abstrakcyjnej klasie BaseEntity, w której zastosowano adnotacje @CreatedDate oraz @LastModifiedDate. Dzięki temu system automatycznie, z poziomu frameworka, zarządza sygnaturami czasowymi tworzenia i edycji rekordów, zdejmując ten obowiązek z warstwy logiki biznesowej.

Tabela: appointments								
	data_modyfikacji	data_utworzenia	date_time	doctor_id	id	patient_id	description	status
	Filtr	Filtr	Filtr	Filtr	Filtr	Filtr	Filtr	Filtr
1	1780214853373	1780214853373	1780300800000	2	2	2	Alergia skórna - pierwsza wizyta	ZAPLANOWANA
2	1780214853379	1780214853379	1780304400000	3	3	3	Szczepienie coroczne	ZAPLANOWANA
3	1780217669447	1780217669447	1780308000000	3	4	4	Kastracja	ZAPLANOWANA
4	1780225034411	1780214853366	1780297200000	1	1	1	Badanie kontrolne po operacji	ODWOLANA

Tabela: appointments								
	data_modyfikacji	data_utworzenia	date_time	doctor_id	id	patient_id	description	status
	Filtr	Filtr	Filtr	Filtr	Filtr	Filtr	Filtr	Filtr
1	2026-05-31 ...	2026-05-31 ...	2026-06-01 ...	2	2	2	Alergia skórna - pierwsza wizyta	ZAPLANOWANA
2	2026-05-31 ...	2026-05-31 ...	2026-06-01 ...	3	3	3	Szczepienie coroczne	ZAPLANOWANA
3	2026-05-31 ...	2026-05-31 ...	2026-06-01 ...	3	4	4	Kastracja	ZAPLANOWANA
4	2026-05-31 ...	2026-05-31 ...	2026-06-01 ...	1	1	1	Badanie kontrolne po operacji	ODWOLANA

Tabela: patients								
	data_modyfikacji	data_utworzenia	id	name	owner_email	owner_name	owner_phone	species
	Filtr	Filtr	Filtr	Filtr	Filtr	Filtr	Filtr	Filtr
1	2026-05-31 ...	2026-05-31 ...	1	Burek	NULL	Jan Kowalski	NULL	Pies
2	2026-05-31 ...	2026-05-31 ...	2	Mruczek	NULL	Maria Nowak	NULL	Kot
3	2026-05-31 ...	2026-05-31 ...	3	Złotka	NULL	Piotr Zając	NULL	Pies
4	2026-05-31 ...	2026-05-31 ...	4	Filemon	NULL	Anna Wróbel	NULL	Kot
5	2026-05-31 ...	2026-05-31 ...	5	TestowyPies	NULL	Mariusz Torpeda	NULL	Pies
6	2026-05-31 ...	2026-05-31 ...	7	TestowyPi...	m.to@mai...	Monika Toczek	712367825	Pies
7	2026-05-31 ...	2026-05-31 ...	6	TestowyPi...	m.pol@ma...	Marianna ...	736836098	Pies

Tabela: doctors						
	data_modyfikacji	data_utworzenia	id	first_name	last_name	specialization
	Filtr	Filtr	Filtr	Filtr	Filtr	Filtr
1	1780214853279	1780214853279	1	Anna	Kowalska	Weterynarz ogólny
2	1780214853328	1780214853328	2	Marek	Nowak	Weterynarz chirurg
3	1780214853334	1780214853334	3	Katarzyna	Wiśniewska	Weterynarz egzotyczny
4	1780219717319	1780219717319	4	Mariusz	Nowak	Weterynarz egzotyczny

Rys.2,3,4,5. Podgląd tabel relacyjnej bazy danych SQLite (wizyty, pacjenci, lekarze).

5. Kluczowa Logika Biznesowa (Reguły Dziedziczne)

Największy stopień złożoności algorytmicznej i logicznej znajduje się w warstwie zarządzania wizytami (AppointmentService). Zaimplementowano w niej rygorystyczne zasady rezerwacji:

1. **Walidacja Czasu Pracy:** Algorytm weryfikuje dzień tygodnia oraz dokładną godzinę, blokując żądania na dni weekendowe oraz godziny poza zakresem 08:00 – 18:00.
2. **Okno Antykolizyjne (Margines 30-minutowy):** Przy użyciu natywnego zapytania zdefiniowanego przez @Query system przeszukuje bazę danych w celu znalezienia wizyt przypisanych do wybranego lekarza. Zapytanie sprawdza bezwzględną różnicę czasu między istniejącymi wizytami a nową wizytą. Jeżeli różnica jest mniejsza niż 30 minut, rzucony jest wyjątek AppointmentConflictException.
3. **Mechanizm Soft-Delete:** Usunięcie wizyty z punktu widzenia użytkownika nie wywołuje fizycznej instrukcji DELETE w bazie danych. Wykorzystano typy wyliczeniowe (Enum) z modyfikacją statusu (z PLANNED na CANCELED), co pozwala na utrzymanie kompletności danych analitycznych.

6. Scentralizowana Obsługa Błędów

W celu zapewnienia zjawiska tzw. "wdzięcznej degradacji" (graceful degradation) oraz ustandaryzowania odpowiedzi dla klienta API, wykorzystano mechanizm

GlobalExceptionHandler z adnotacją @RestControllerAdvice. Przechwytuje on wyjątki aplikacyjne i tłumaczy je na zestandaryzowane komunikaty JSON z odpowiednimi kodami HTTP (400 Bad Request przy błędnej walidacji numeru telefonu czy adresu e-mail, 404 Not Found dla braku zasobów oraz 409 Conflict dla naruszeń logiki czasu).

7. Interfejs Użytkownika (Frontend)

Graficzny interfejs powstał jako lekka aplikacja typu SPA. Do manipulacji drzewem DOM użyto czystego języka JavaScript. Widoki dynamicznie renderują odpowiednie komunikaty barwne (np. zielone dla wizyt zaplanowanych, czerwone dla odwołanych), znacznie ułatwiając orientację w kalendarzu.

Appointments List

ID: 1 | 2026-06-01 09:00:00 CANCELED
Doctor: Anna Kowalska | Patient: Burek (Jan Kowalski)
Desc: Badanie kontrolne po operacji

ID: 2 | 2026-06-01 10:00:00 PLANNED
Doctor: Marek Nowak | Patient: Mruczek (Maria Nowak)
Desc: Alergia skórna – pierwsza wizyta

ID: 3 | 2026-06-01 11:00:00 PLANNED
Doctor: Katarzyna Wiśniewska | Patient: Złotka (Piotr Zając)
Desc: Szczepienie coroczne

ID: 4 | 2026-06-01 12:00:00 PLANNED
Doctor: Katarzyna Wiśniewska | Patient: Filemon (Anna Wróbel)
Desc: Kastracja

Rys. 6. Graficzny interfejs użytkownika (klient webowy) – widok panelu zarządzania harmonogramem z oznaczaniem statusów wizyt.

8. Zapewnienie Jakości (Automatyzacja Testów)

Niezawodność reguł biznesowych została zweryfikowana poprzez implementację 36 zautomatyzowanych testów jednostkowych. Rozkład scenariuszy badawczych prezentuje się następująco:

- **AppointmentServiceTest (14 testów):** Rygorystyczne testowanie stanów brzegowych (np. rezerwacja dokładnie na styku 30 minut, rejestracja dokładnie o 18:00).
- **DoctorServiceTest (10 testów):** Weryfikacja operacji odczytu, zapisu oraz algorytmów filtrujących po specjalizacjach medycznych.
- **PatientServiceTest (12 testów):** Sprawdzanie mechanizmów obsługi braku danych kontaktowych (wartości null) oraz poprawności wstrzykiwania zależności przy operacjach modyfikujących.

```
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.667 s -- in com.veterinary.clinic.services.AppointmentServiceTest
[INFO] Running com.veterinary.clinic.services.DoctorServiceTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.131 s -- in com.veterinary.clinic.services.DoctorServiceTest
[INFO] Running com.veterinary.clinic.services.PatientServiceTest
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.096 s -- in com.veterinary.clinic.services.PatientServiceTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 36, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.935 s
[INFO] Finished at: 2026-05-31T14:46:17+02:00
[INFO] -----
```

Rys. 7. Raport z wykonania zautomatyzowanych testów jednostkowych w terminalu, potwierdzający pomyślne przejście 36 scenariuszy.